

SS2 1 Search Engine Functional Requirements

Design a search engine for boolean retrieval

How do you build a search system on the 37 plays of Shakespeare that handles Boolean queries of the form:

Caesar

Brutus AND Caesar

Caesar OR Cleopatra

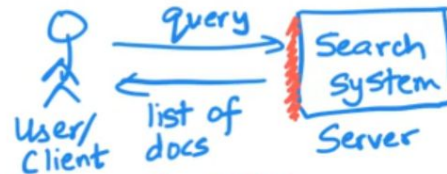
NOT Calpurnia

No free text queries

united states presidential election results 2020

No ranking

Given: A collection of documents (corpus),
a user with an information need,
expressed as a query

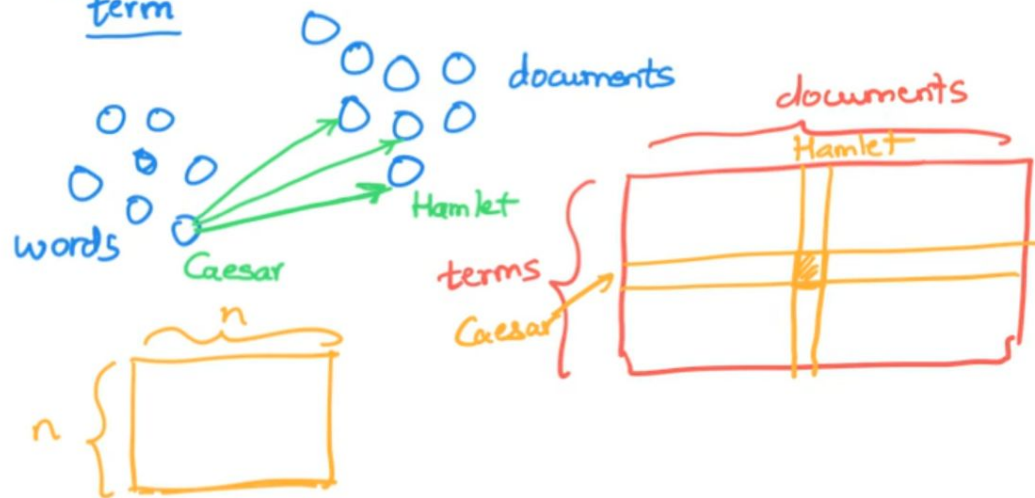


Naive solution: Something like
grep?
v. slow!
 $O(\text{corpus size})$

SS2 2 Term-Document Incidence Matrix

Transform and conquer : change the input representation

word/term → documents



Brutus AND Caesar
NOT Calpurnia

Brutus AND Caesar BUT NOT
Calpurnia

1 if play contains
word, 0 otherwise

- So we have a 0/1 vector for each term.
- To answer query: take the vectors for **Brutus**, **Caesar** and **Calpurnia** (complemented) → bitwise AND.
 - 110100 AND
 - 110111 AND
 - 101111 =
 - 100100

	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth
Antony	1	1	0	0	0	1
Brutus	1	1	0	1	0	0
Caesar	1	1	0	1	1	1
Calpurnia	0	1	0	0	0	0
Cleopatra	1	0	0	0	0	0
mercy	1	0	1	1	1	1
worser	1	0	1	1	1	0

creep	1	0	1	1	1	1
mercy	1	0	1	1	1	1
worse	1	0	1	1	1	0

Answers to query

- Antony and Cleopatra, Act III, Scene ii

Agrippa [Aside to *DOMITIUS ENOBARBUS*]: Why, *Enobarbus*,
 When Antony found Julius **Caesar** dead,
 He cried almost to roaring; and he wept
 When at Philippi he found **Brutus** slain.

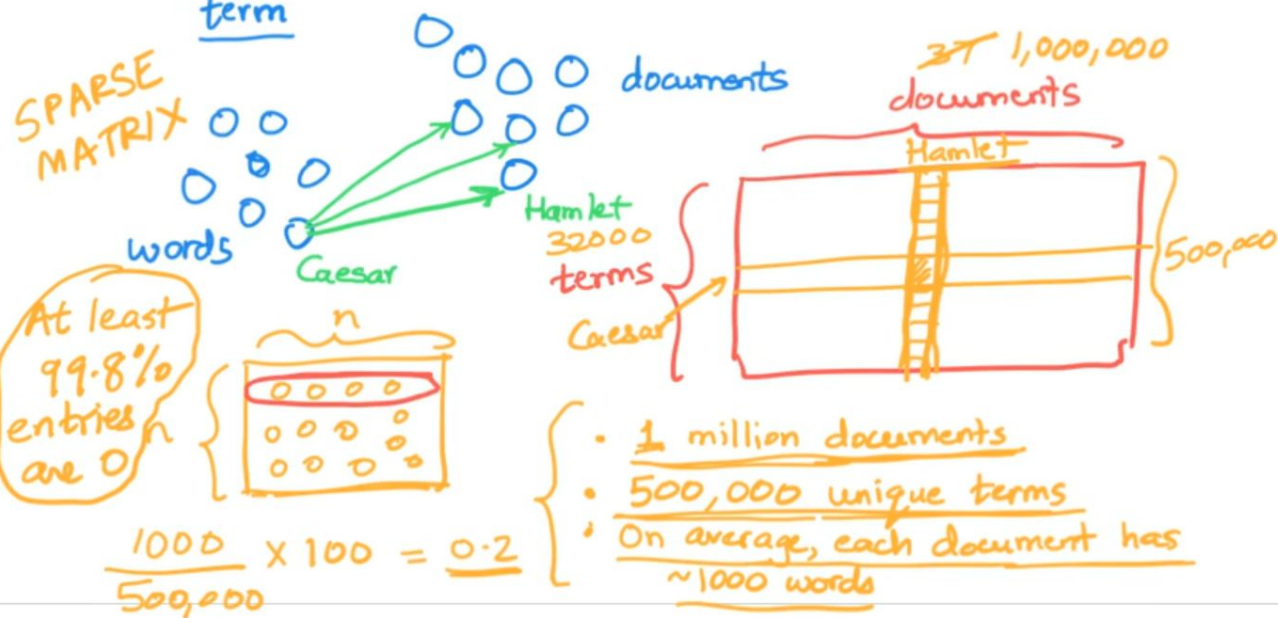
- Hamlet, Act III, Scene ii

Lord Polonius: I did enact Julius **Caesar** I was killed i' the
 Capitol; **Brutus** killed me.

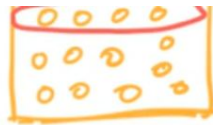


Transform and conquer: change the input representation

word/term → documents



entries
are 0



$$\frac{1000}{500,000} \times 100 = 0.2$$

- 1 million documents
- 500,000 unique terms
- On average, each document has ~1000 words

Term-document incidence matrices

	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth
Antony	1	1	0	0	0	1
Brutus	1	1	0	1	0	0
Caesar	1	1	0	1	1	1
Calpurnia	0	1	0	0	0	0
Cleopatra	1	0	0	0	0	0
mercy	1	0	1	1	1	1
worser	1	0	1	1	1	0

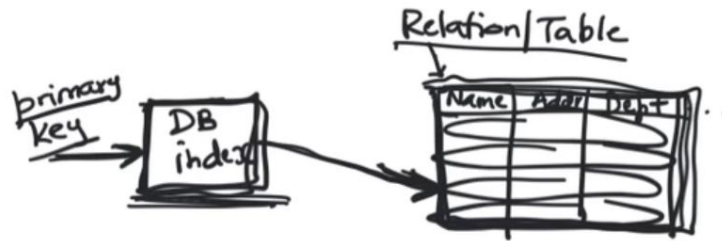
101111

Query : Caesar OR
Brutus AND Caesar
NOT Calpurnia

Brutus AND Caesar BUT NOT
Calpurnia

1 if play contains
word, 0 otherwise

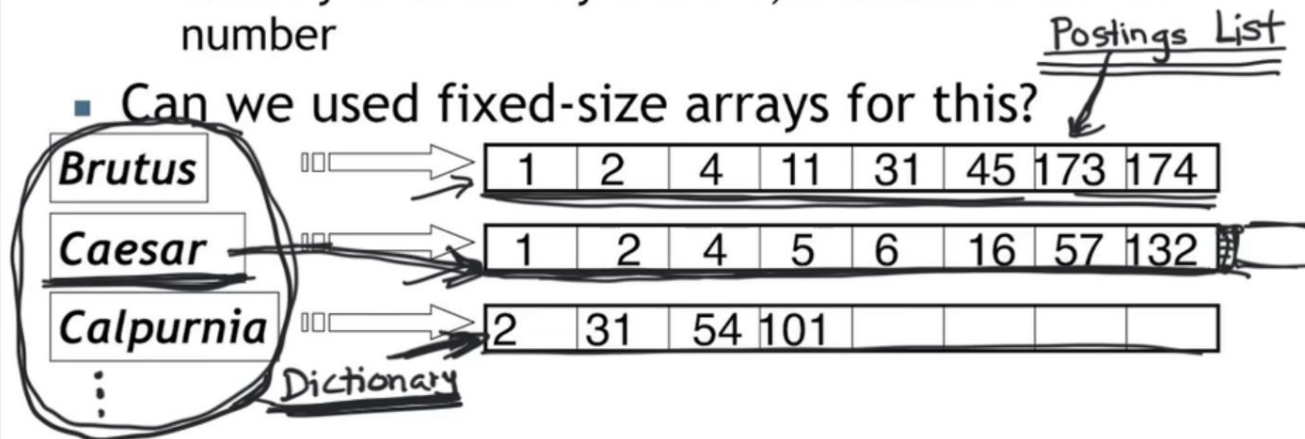
SS2 3 Inverted Index



Inverted index

- For each term t , we must store a list of all documents that contain t .
 - Identify each doc by a **docID**, a document serial number

- Can we use fixed-size arrays for this?

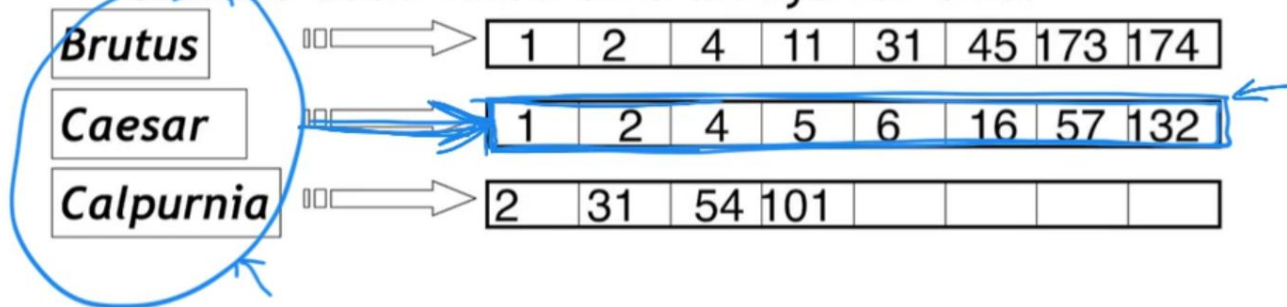


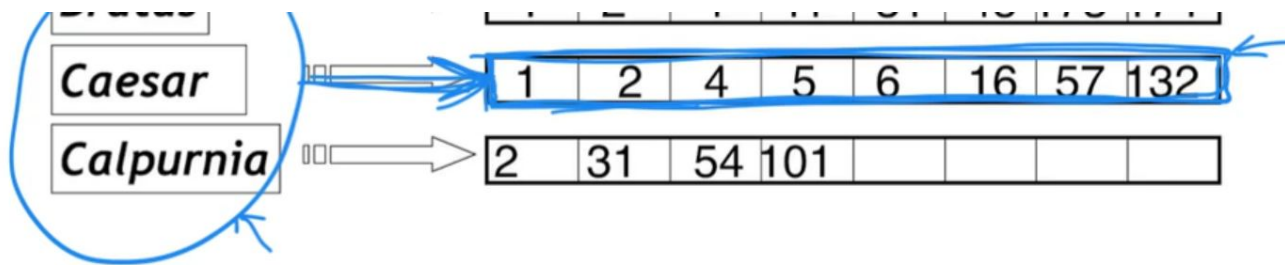
SS2 4 In-memory Index Construction and Serving Search Results

Query : Caesar

Inverted index

- For each term t , we must store a list of all documents that contain t .
 - Identify each doc by a **docID**, a document serial number
- Can we use fixed-size arrays for this?



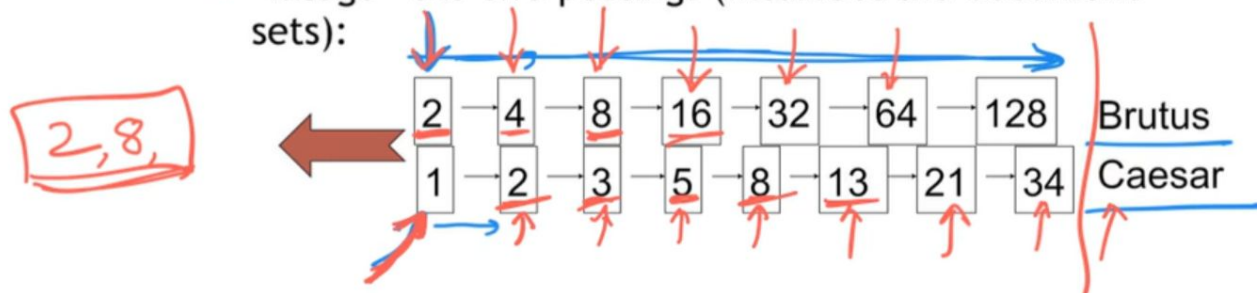


Query processing: AND

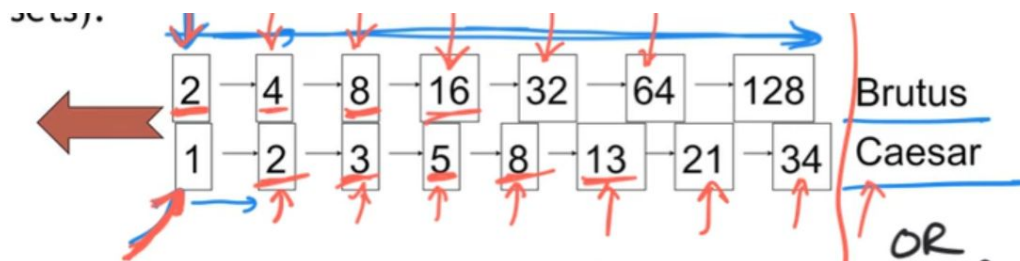
- Consider processing the query:

Brutus AND Caesar

- Locate Brutus in the Dictionary;
 - Retrieve its postings.
- Locate Caesar in the Dictionary;
 - Retrieve its postings.
- "Merge" the two postings (intersect the document sets):



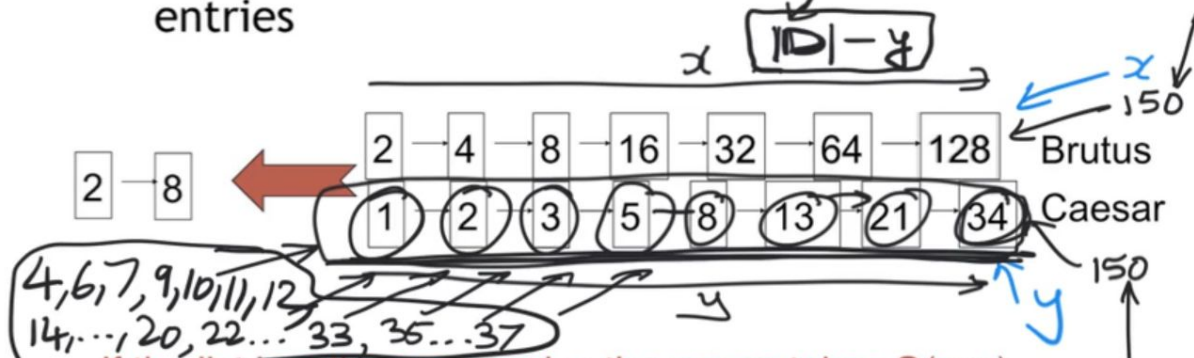
2, 8,



Query: Caesar AND Brutus
~~NOT Caesar~~

The merge

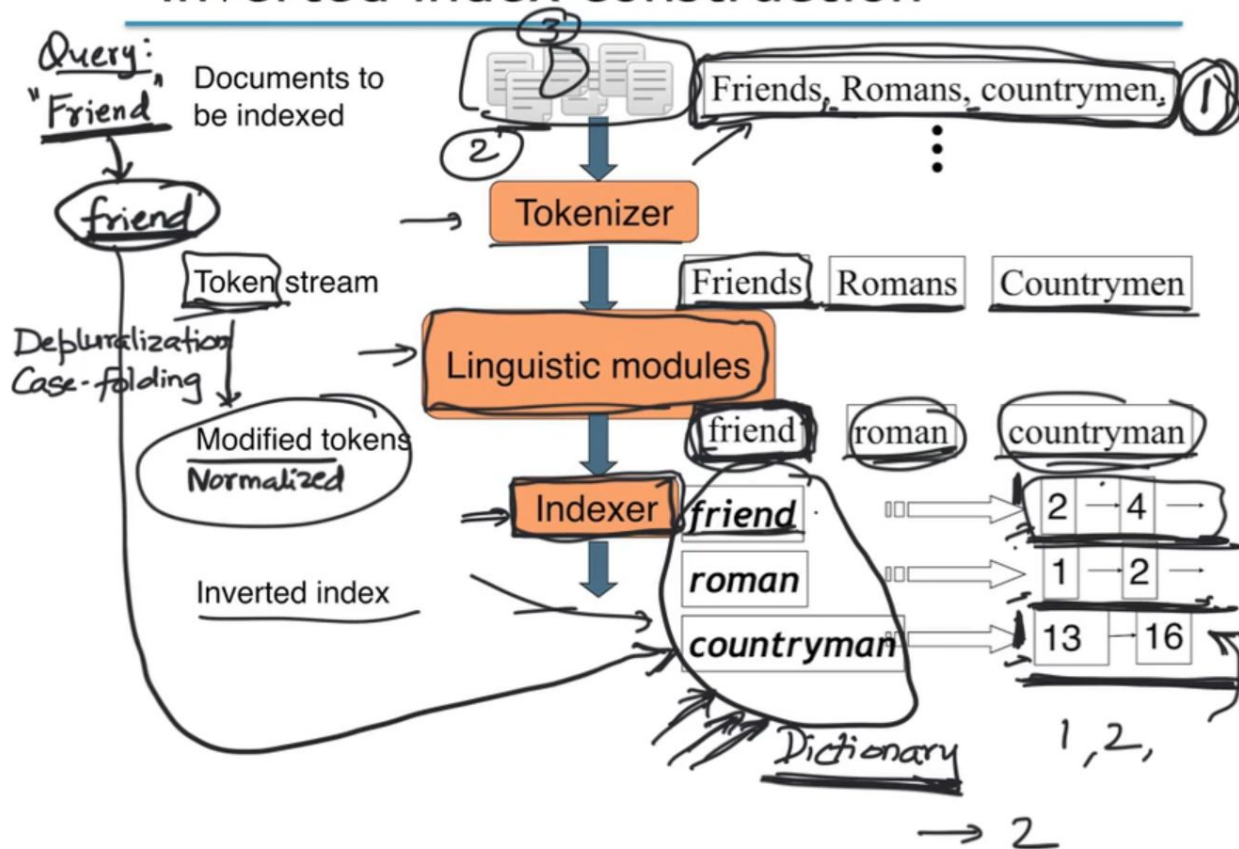
- Walk through the two postings simultaneously, in time linear in the total number of postings entries



If the list lengths are x and y , the merge takes $O(x+y)$ operations.

Crucial: postings sorted by docID.

Inverted index construction



- How to construct an inverted index (in-memory)
- How to answer AND, OR, NOT Boolean queries + time complexity

2, 8

- "Merge" the sets):



The me

- Walk through time line entries

2 — 8

4, 6, 7, 9, 10, 11, 12, 14, ..., 20, 22, ...

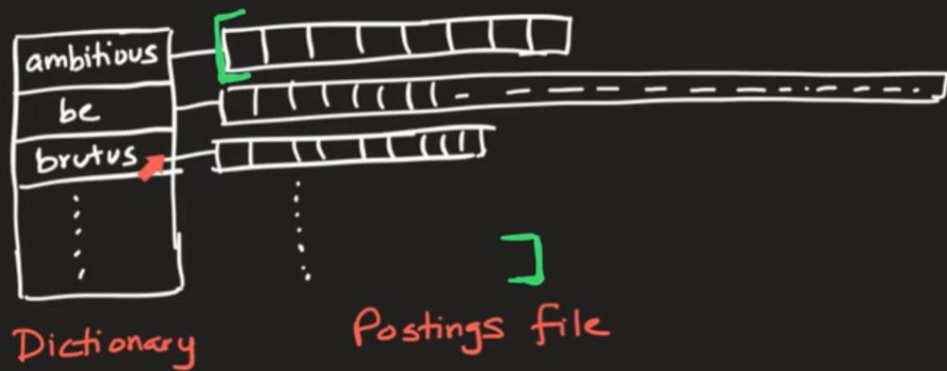
If the list length operations.

Crucial: position

{ik}

SS2 5 Inverted Index Construction on Disk

Suppose the inverted index is disk-based.
How do we construct it?



If I could get all (term, doc ID) pairs
in sorted order on disk, I could scan
those pairs and build out each postings list.

Indexer steps: Token sequence

- Sequence of (Modified token, Document ID) pairs.

Doc 1

I did enact Julius
Caesar I was killed
i' the Capitol;
Brutus killed me.

Doc 2

So let it be with
Caesar. The noble
Brutus hath told you
Caesar was ambitious



Term	docID
I	1
did	1
enact	1
julius	1
caesar	1
I	1
was	1
killed	1
i'	1
the	1
capitol	1
brutus	1
killed	1
me	1
so	2
let	2
it	2
be	2
with	2
caesar	2
the	2
noble	2
brutus	2
hath	2
told	2
you	2
caesar	2
was	2
ambitious	2

Indexer steps: Sort

- Sort by terms
 - At least conceptually




Core indexing step

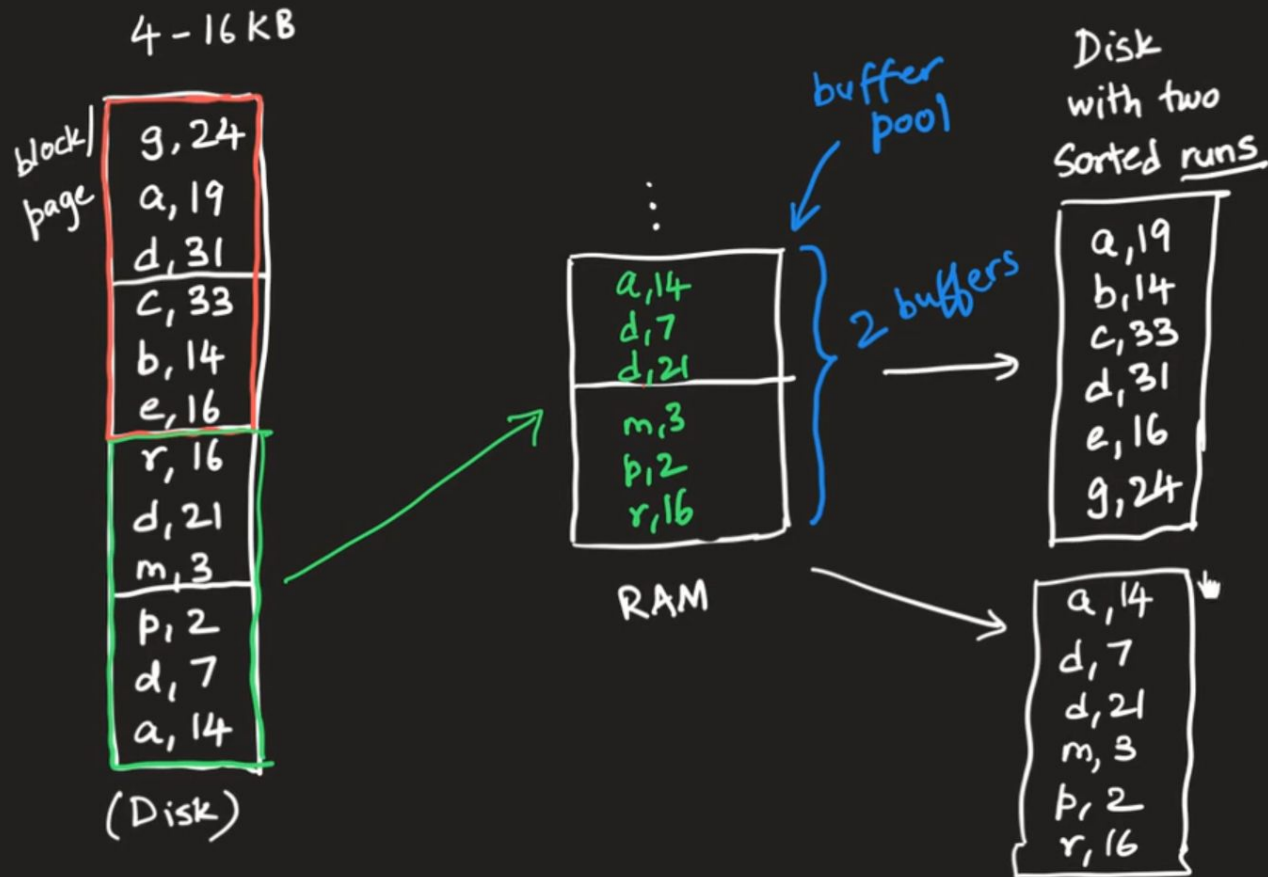
Internal Sort vs External Sort

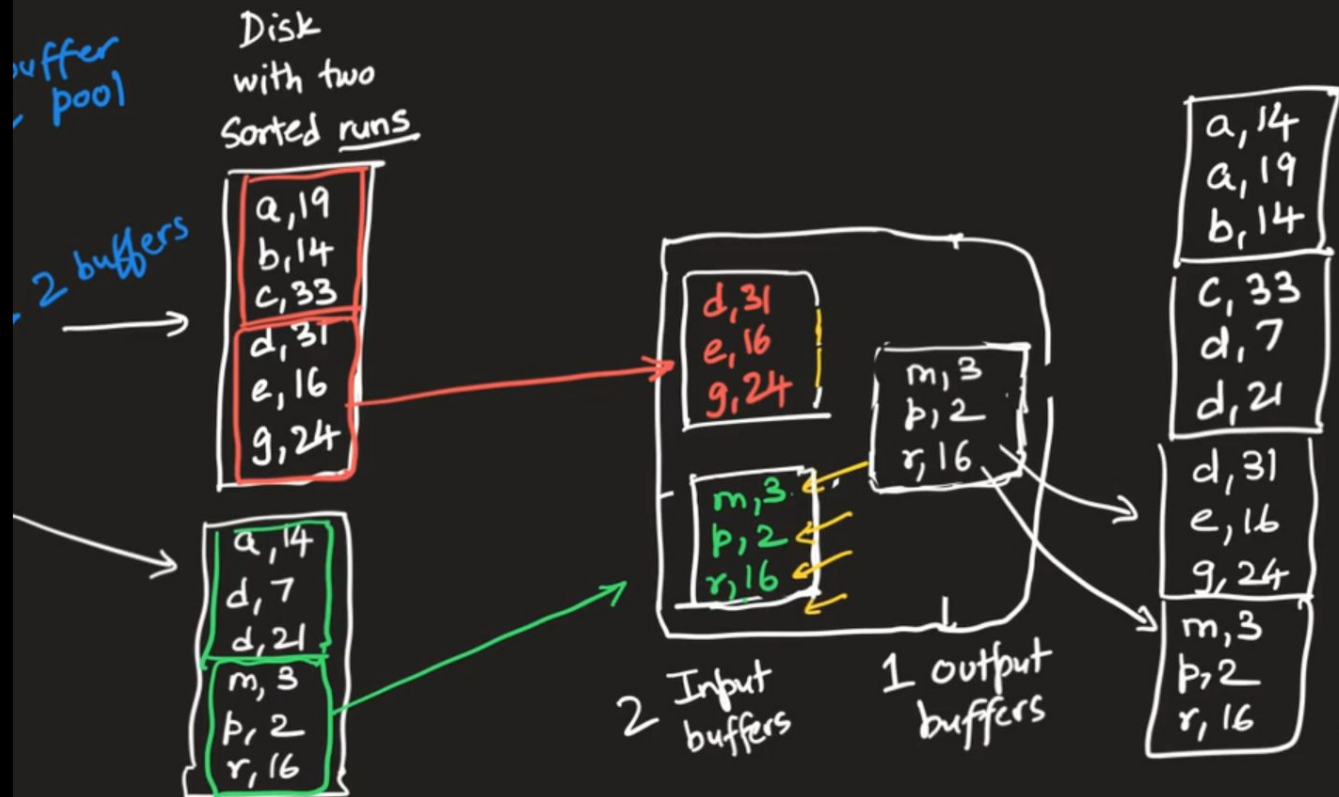
Term	docID
I	1
did	1
enact	1
julius	1
caesar	1
I	1
was	1
killed	1
i'	1
the	1
capitol	1
brutus	1
killed	1
me	1
so	2
let	2
it	2
be	2
with	2
caesar	2
the	2
noble	2
brutus	2
hath	2
told	2
you	2
caesar	2
was	2
ambitious	2

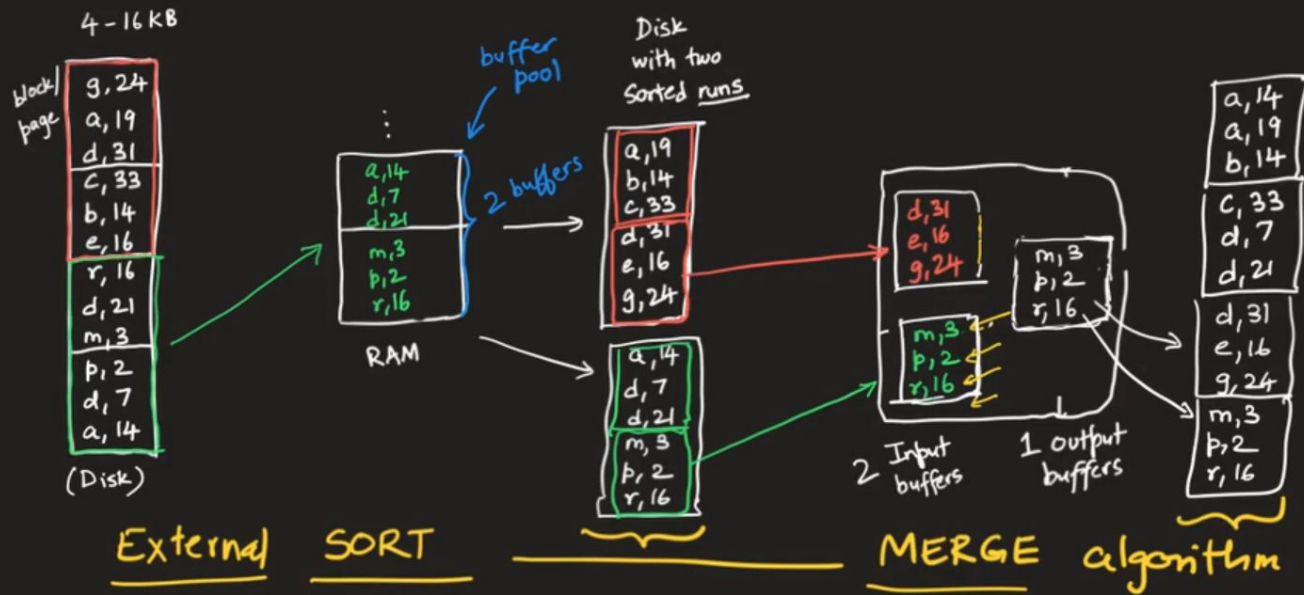


Term	docID
ambitious	2
be	2
brutus	1
brutus	2
capitol	1
caesar	1
caesar	2
caesar	2
did	1
enact	1
hath	1
I	1
I	1
i'	1
it	2
julius	1
killed	1
killed	1
let	2
me	1
noble	2
so	2
the	1
the	2
told	2
you	2
was	1
was	2
with	2

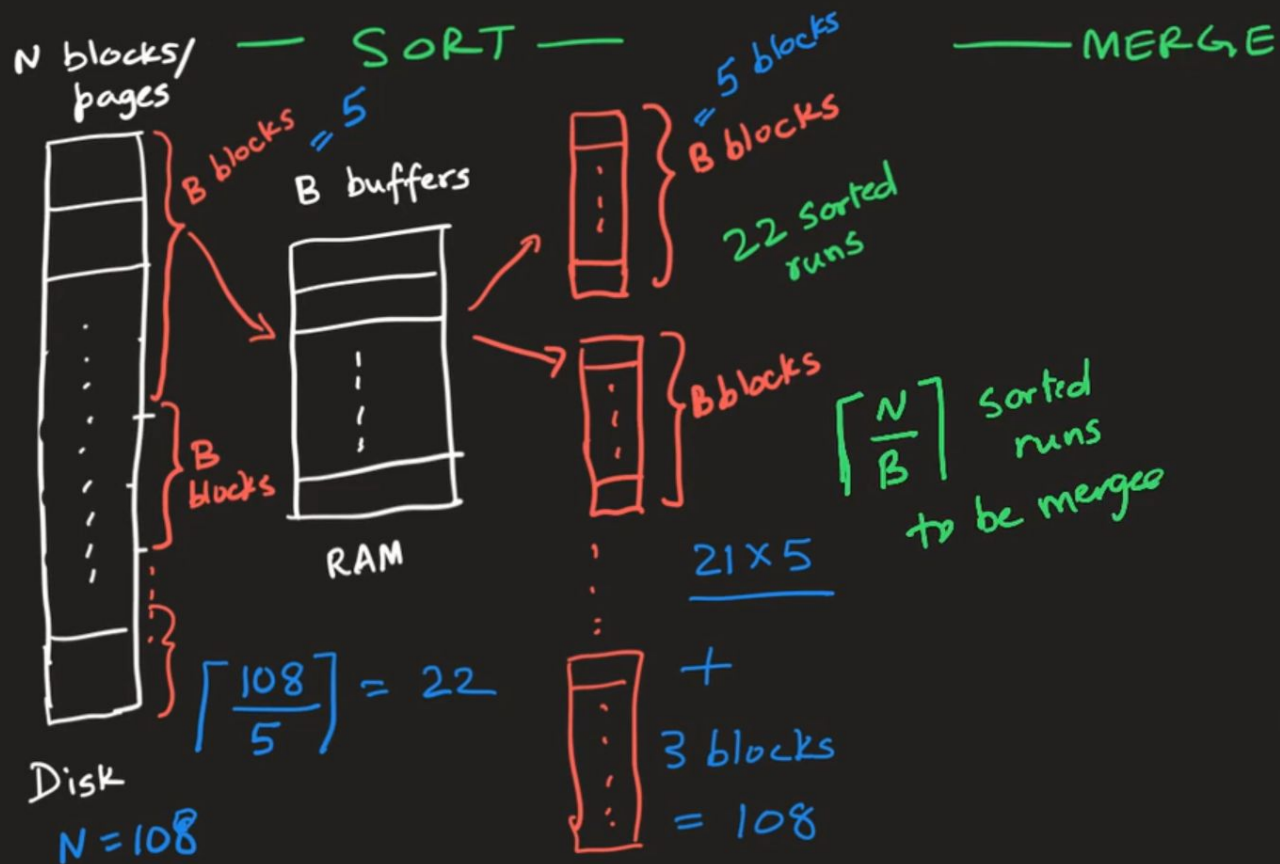
SS2 6 External Sort-Merge Example

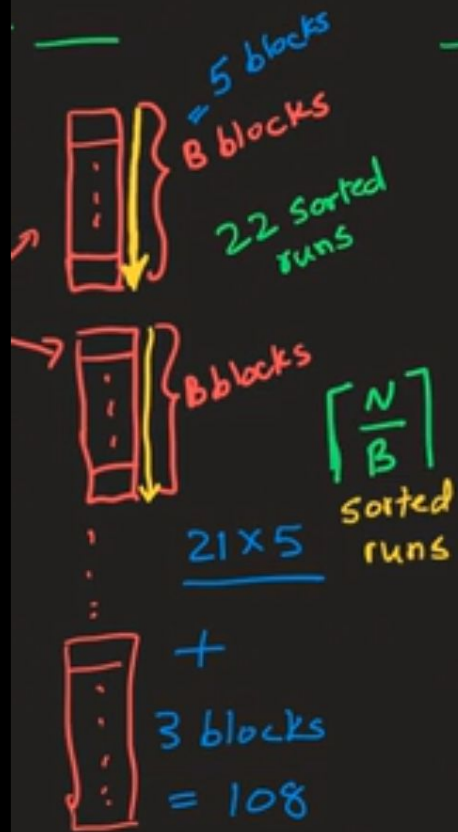






SS2 7 K-way External Sort-Merge





MERGE



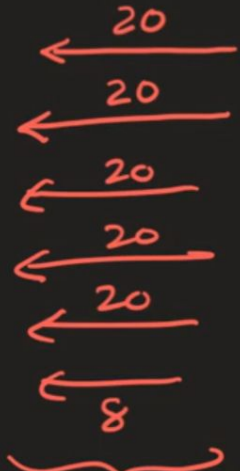
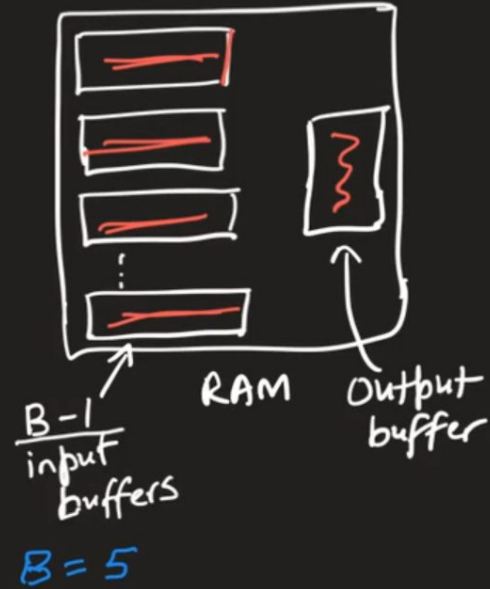
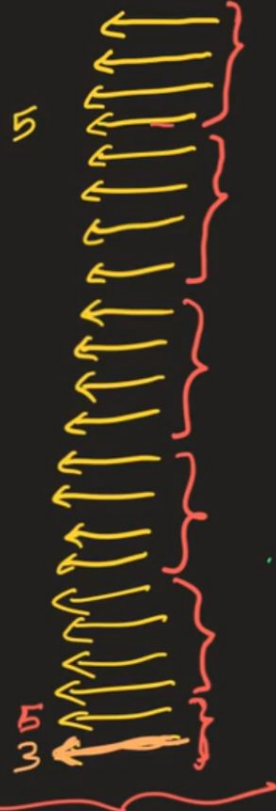
$B = 5$

MERGE

sorted
runs

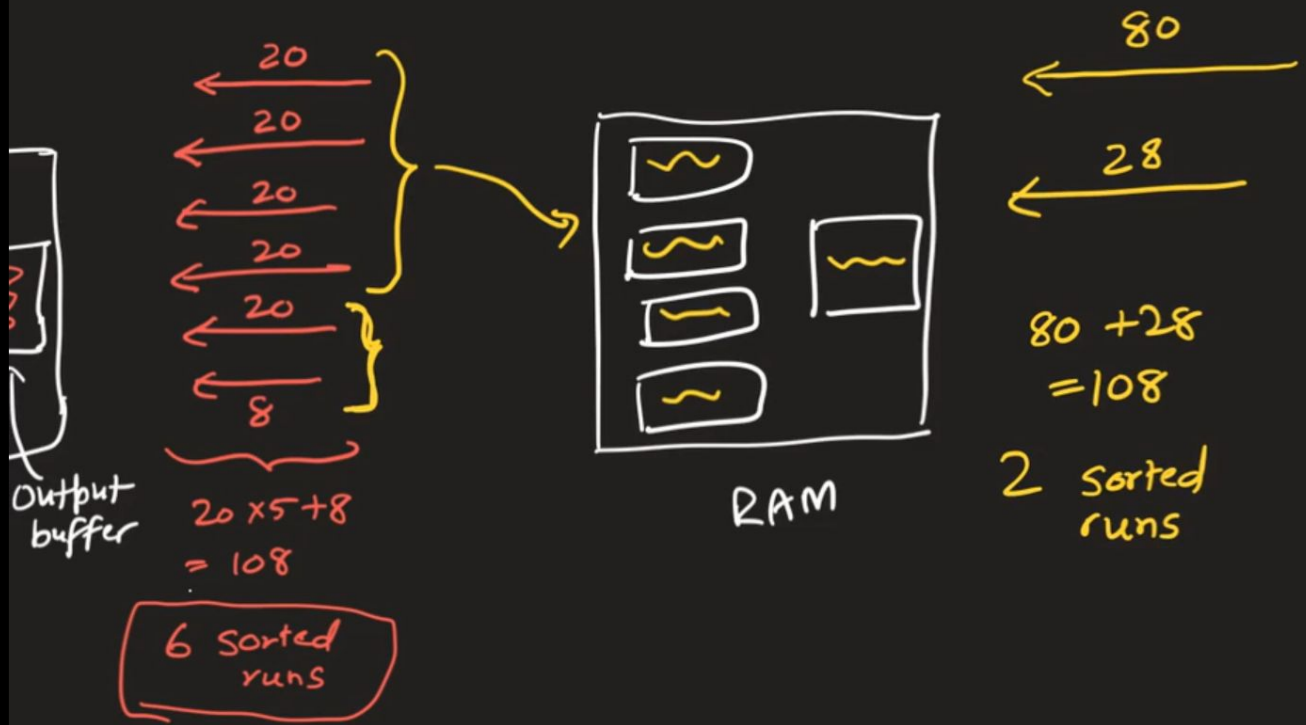
$$\left\lceil \frac{N}{B} \right\rceil$$

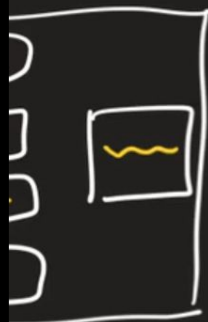
sorted
runs



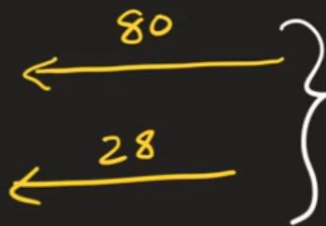
$$20 \times 5 + 8 = 108$$

6 sorted
runs





RAM

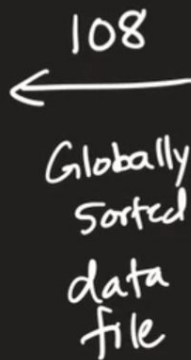


$$80 + 28 = 108$$

2 sorted runs



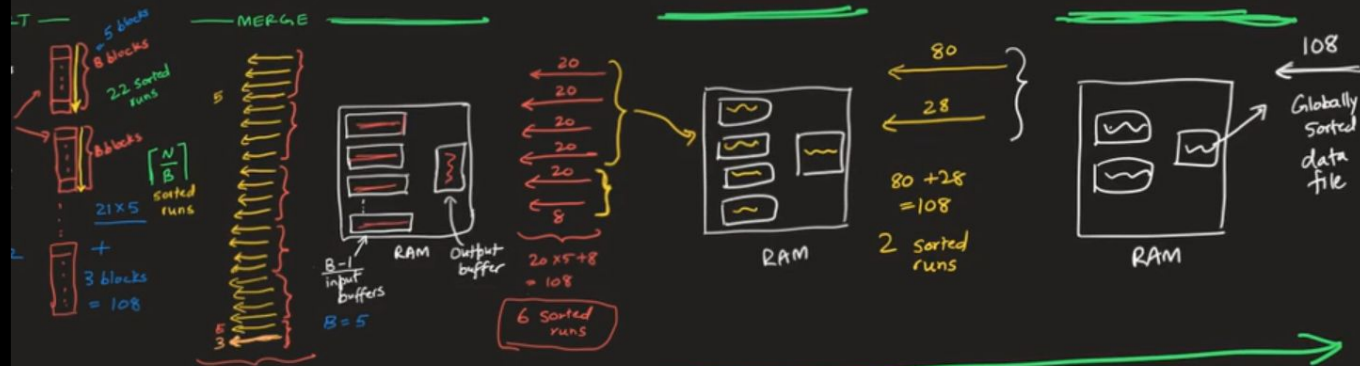
RAM



$$i = \log_{B-1} \left(\frac{N}{B} \right) \quad \frac{N}{B} = 1 \Rightarrow \frac{N}{B} = (B-1)^i$$

passes in merge phase

$$\log_{B-1} \left(\frac{N}{B} \right) = i \underbrace{\log_{B-1} (B-1)}_1$$



$$\left\lceil \frac{N}{B} \right\rceil \rightarrow \left\lceil \frac{\left\lceil \frac{N}{B} \right\rceil}{B-1} \right\rceil \rightarrow \frac{N}{B} \rightarrow \frac{N}{(B-1)^3}$$

$$\left(\frac{N}{B} \right) \rightarrow \frac{N}{(B-1)^2}$$

$$\log_4(16) = 2 \dots$$

$$21.6$$

$$2 \times 108 \times (3+1) = 108 \times 8 = 864 \text{ ops.}$$

$$\# \text{ disk I/O ops} = \frac{\text{Ops in SORT}}{2N} + \frac{\text{Ops in merge}}{2Ni}$$

$$\log_4\left(\frac{108}{5}\right) = \log_4(21.6)$$

$$= \frac{2N(i+1)}{2N}$$

$$i \times \underbrace{(N+N)}_{\substack{\text{read} \\ \text{write}}}$$

$$i = \left\lceil \log_{B-1}\left(\frac{N}{B}\right) \right\rceil$$

$$\frac{N}{B} = (B-1)^i$$

passes in merge phase

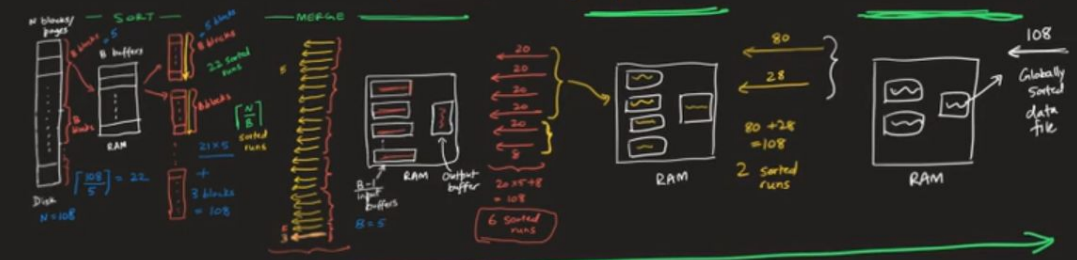
$$= 1 \Rightarrow \frac{N}{B} = (B-1)^i$$

$$\log_{B-1}\left(\frac{N}{B}\right) =$$



$\log_4(108) = 2 \dots$
 $\log_4\left(\frac{108}{5}\right) \approx \log_4(21.6)$
 $i = \left\lceil \log_{B-1}\left(\frac{N}{B}\right) \right\rceil = 3$
 $\# \text{ passes in merge phase} = i$
 $\# \text{ disk IO ops} = \underbrace{\log_4\left(\frac{108}{5}\right)}_{21.6} + \underbrace{2 \times 108 \times (3+1)}_{108 \times 8 = 864 \text{ ops.}} = 885.6$
 $\# \text{ disk IO ops} = \underbrace{2Ni}_{2 \times 108 \times 3 = 648} + \underbrace{i \times (N+N)}_{3 \times 216 = 648}$
 $\frac{N}{B} = (B-1)^i \Rightarrow \log_{B-1}\left(\frac{N}{B}\right) = i$
 $i \times (N+N)$ (read write)

K-way External Sort-Merge Algorithm

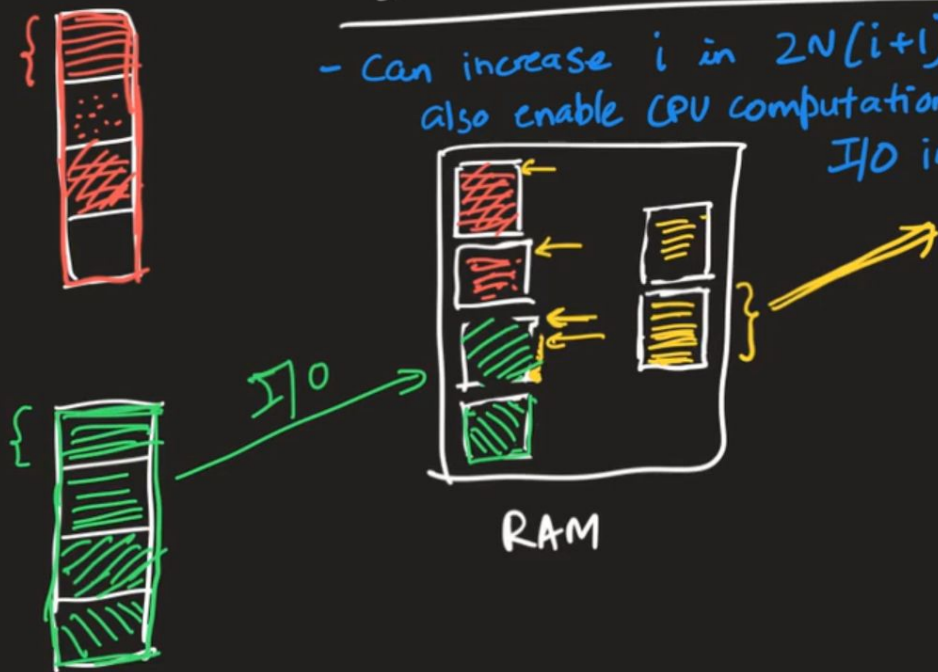


$\left\lceil \frac{N}{B} \right\rceil \rightarrow \left\lceil \frac{\left\lceil \frac{N}{B} \right\rceil}{B-1} \right\rceil \rightarrow \frac{N}{B} \rightarrow \frac{N}{B} \cdot \frac{1}{(B-1)^2} \rightarrow \frac{N}{B} \cdot \frac{1}{(B-1)^3}$

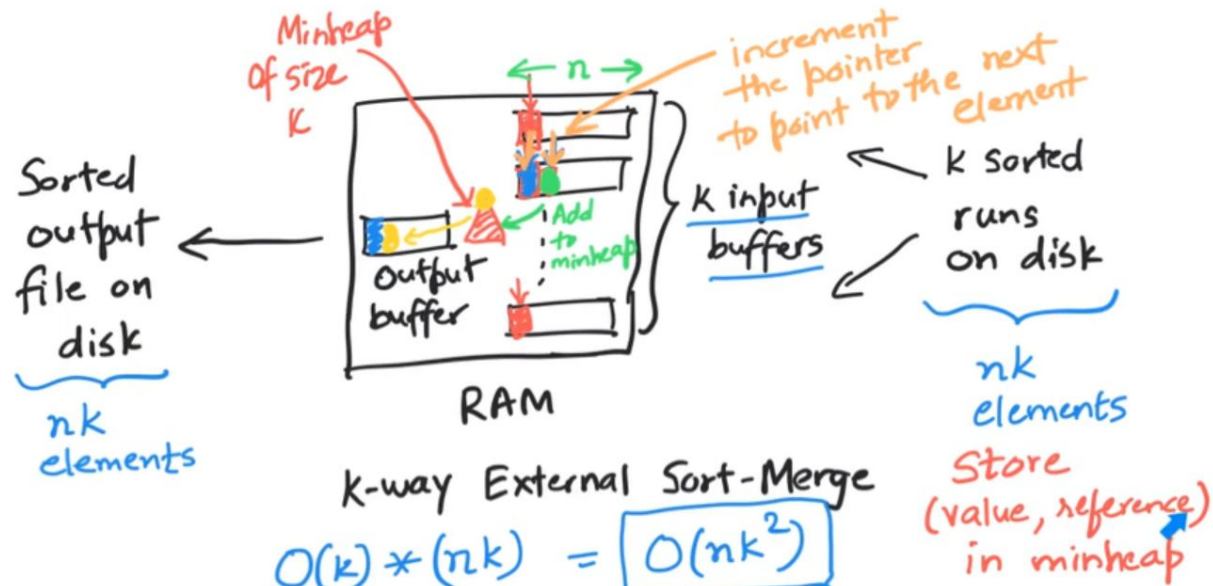
SS2 8 Double Buffering (optional)

DOUBLE BUFFERING

- Can increase i in $2N(i+1)$ but also enable CPU computation and disk I/O in parallel



SS2 9 Merge K Sorted Lists



k-way External Sort-Merge

$$O(k) * (nk) = O(nk^2)$$

Operation to be optimized : Finding the min among k values

Minheap with k values

Extract-min $O(\log k)$ + Insert

$$O(\log k) * nk = O(nk \log k)$$

$$\underline{O(\log k)} * \underline{nk} \text{ in minheap} \\ = O(nk \log k)$$

23. Merge k Sorted Lists

Hard 5821 318 Add to List Share

You are given an array of k linked-lists `lists`, each linked-list is sorted in ascending order.

Merge all the linked-lists into one sorted linked-list and return it.

Example 1:

Input: `lists = [[1,4,5],[1,3,4],[2,6]]`

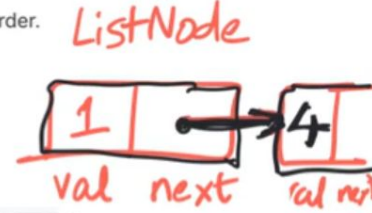
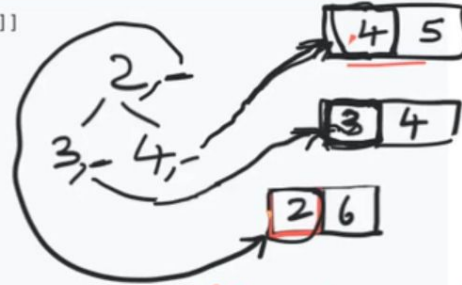
Output: `[1,1,2,3,4,4,5,6]`

Explanation: The linked-lists are:

[
1 → 4 → 5,
1 → 3 → 4,
2 → 6
]

merging them into one sorted list:

1 → 1 → 2 → 3 → 4 → 4 → 5 → 6



(1, reference to the linked list node)

array of k linked lists

function mergeKLists(lists):

minheap = an empty priority queue

for i in 0 to length(lists)-1:

// lists[i] is a ptr to the head of the i th list

if lists[i] is not null:

minheap.insert((lists[i].val, lists[i]))

// Initialize output list

head = new ListNode("sentinel")

tail = head

// Repeatedly extract the min element from heap & append head tail it to the output list

$O(k) * (nk) = O(nk \log k)$

operation to be optimized: Finding the min among k values

Minheap with k values $O(\log k) + \text{Insert}$

$O(\log k) * nk = O(nk \log k)$

23. Merge k Sorted Lists

Hard 5821 318 Add to List Share

You are given an array of k linked-lists lists, each linked-list is sorted in ascending order. Merge all the linked-lists into one sorted linked-list and return it.

Example 1:

Input: lists = [[1,4,5],[1,3,4],[2,6]]

Output: [1,1,2,3,4,4,5,6]

Explanation: The linked-lists are:

1->4->5,
1->3->4,
2->6

merging them into one sorted list:
1->1->2->3->4->4->5->6



(1, reference to list)



while minheap is not empty:

$(val, p) \leftarrow \text{minheap.extractmin}()$

// Append p to the end of the output list

$tail.next = p$

$tail = p$

$p = p.next$

$tail.next = null$

if p is not null:

$\text{minheap.insert}((p.val, p))$

→ $head = head.next$
return head

$O(nk \log k)$

function mergeKlists (lists):

$\text{minheap} = \text{an empty priority queue}$

for i in 0 to $\text{length}(\text{lists}) - 1$:

// $\text{lists}[i]$ is a ptr to the head of the i th list

if $\text{lists}[i]$ is not null:
 $\text{minheap.insert}((\text{lists}[i].val, \text{lists}[i])$

// Initialize output list

$head = \text{new ListNode}("sentinel")$

$tail = head$

// Repeatedly extract the min element from heap & append head tail it to the output list

in minheap
file on disk
nk

output buffer
RA

k-way E

$O(k) * (nk)$

operation to be opt

Minheap with k values

$O(\log)$

23. Merge k Sorted Lists

Hard 5821 318 Add to List

You are given an array of k linked-lists lists .

Merge all the linked-lists into one sorted linked

Example 1:

Input: $\text{lists} = [1, 4, 5], [1, 3, 4], [2, 6]$

Output: $[1, 1, 2, 3, 4, 4, 5, 6]$

Explanation: The linked-lists are:

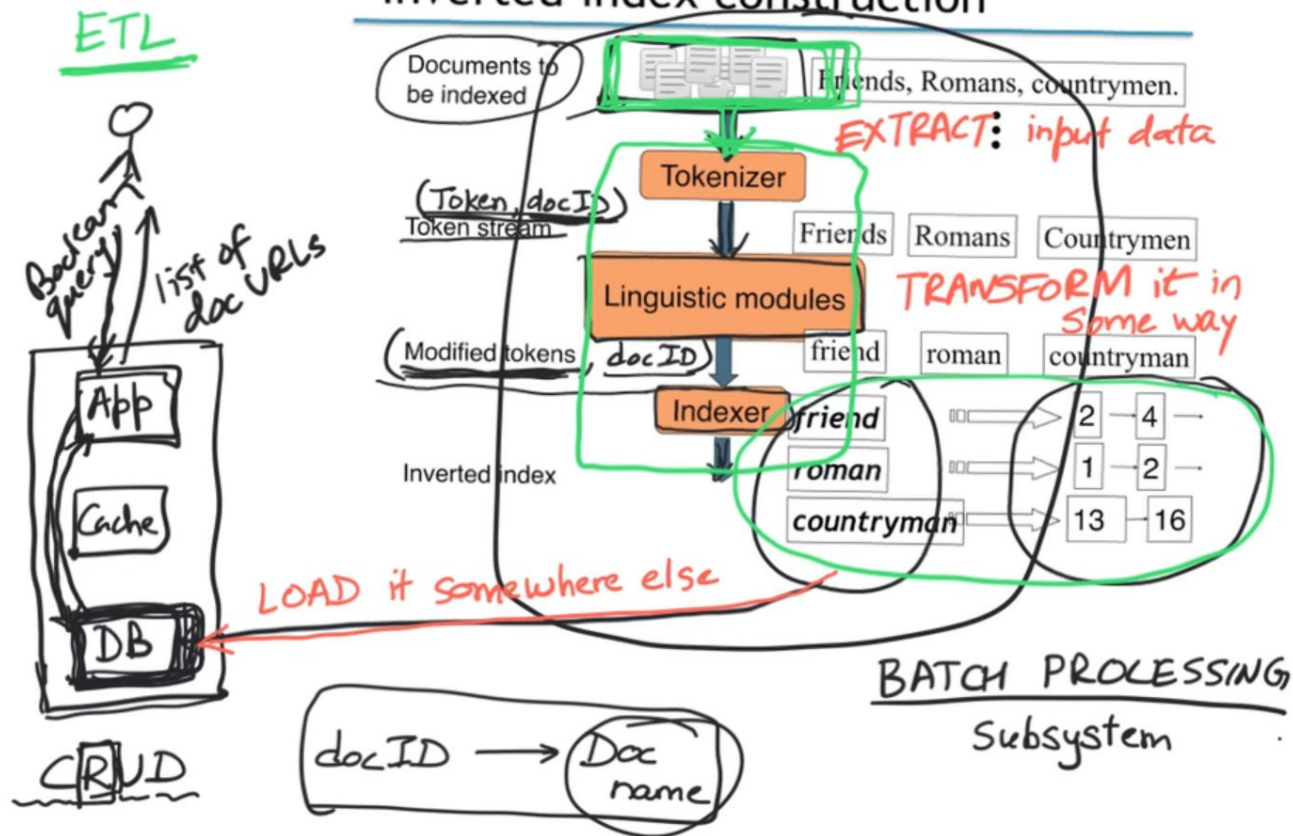
1 → 4 → 5, 1 → 3 → 4, 2 → 6

Merging them into one sorted list:

1 → 1 → 2 → 3 → 4 → 4 → 5 → 6

SS2 10 Batch Processing or ETL

Inverted index construction



SS2 11 SSTable

Dictionary in RAM

⋮
brutal
~~brute~~
~~brutus~~
⋮

Postings file on disk

⋮
brutal : 1, 5, 7, 9, 10, ...
brute : 2, 7, 109, 350, ...
brutus : 108, 150, 153, ...
⋮

} Sorted String
Table
(SSTable)

Query : brut*

brutal OR brute OR brutus OR ...

SS2 12 Distributed File System



The size of the World Wide Web (The Internet)

The Indexed Web contains at least **5.46 billion pages** (Tuesday, 24 November, 2020).

The Dutch Indexed Web contains at least **2753.52 million pages** (Tuesday, 24 November, 2020).

The Indexed Web | [The Dutch Indexed Web](#)

80

Tweet

Like

Share

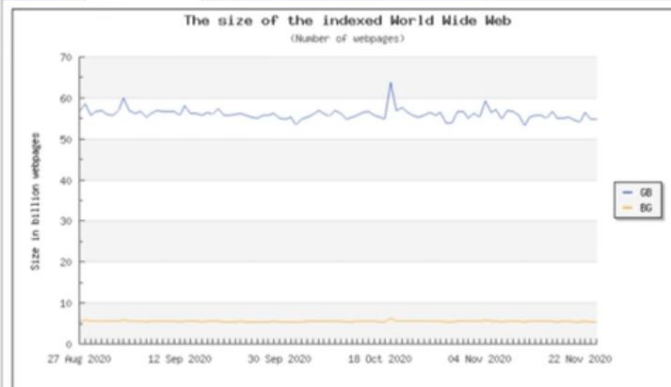
Last Month

Last Three Months

Last Year

Last Two Years

Last Five Years



GB = Sorted on Google and Bing

BG = Sorted on Bing and Google

27 Aug 2020 12 Sep 2020 30 Sep 2020 18 Oct 2020 04 Nov 2020 22 Nov 2020

GB = Sorted on Google and Bing
BG = Sorted on Bing and Google

INDEXER

→ First scale the batch processing pipeline for constructing the index

→ Then scale the online search microservice

RETRIEVER

70,000

Google does not share their search volume data. However, it is estimated Google processes approximately 70,000 search queries every second, translating to 5.8 billion searches per day and approximately **2 trillion** global searches per year. The average person conducts between three and four searches each day. Nov 7, 2019

[blog.hubspot.com](https://blog.hubspot.com/marketing/google-search-statistics) › marketing › google-search-statistics

25 Google Search Statistics to Bookmark ASAP - HubSpot Blog

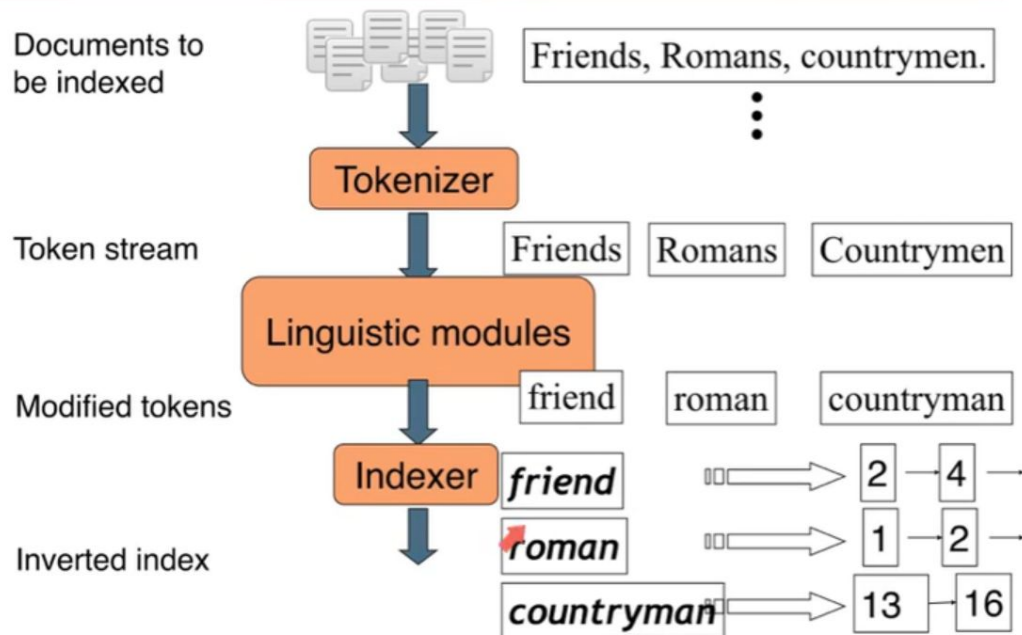


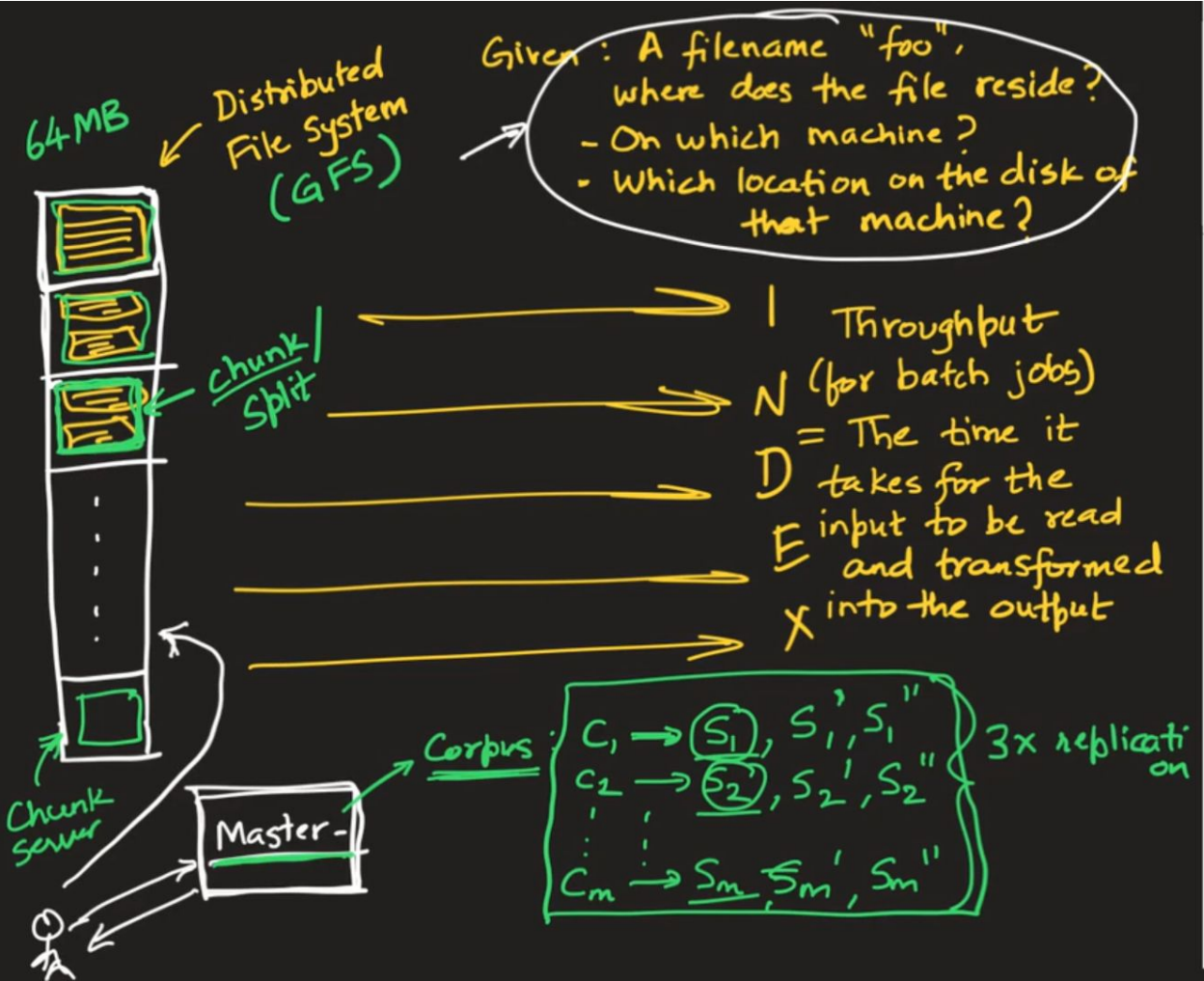
About Featured Snippets



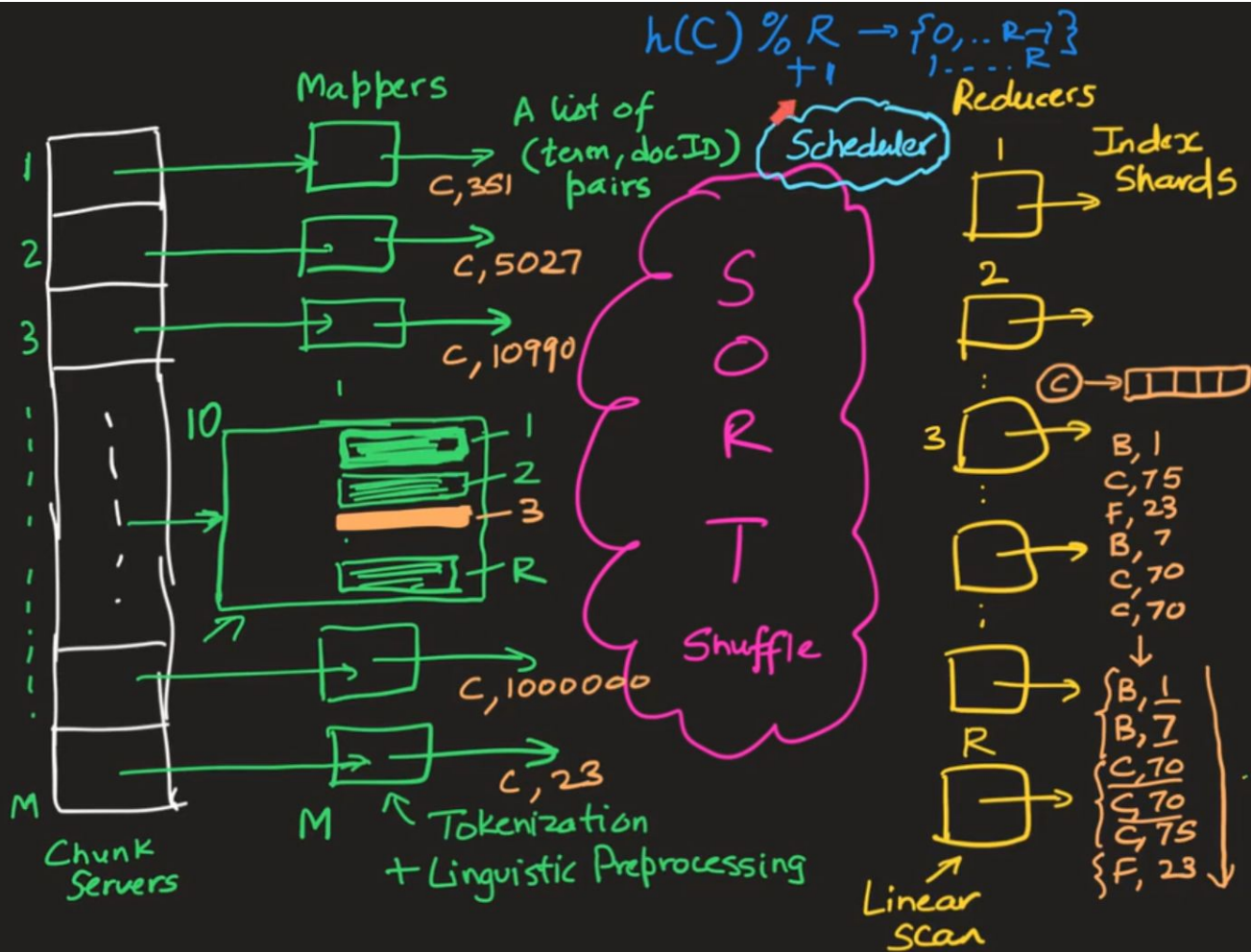
Feedback

Inverted index construction





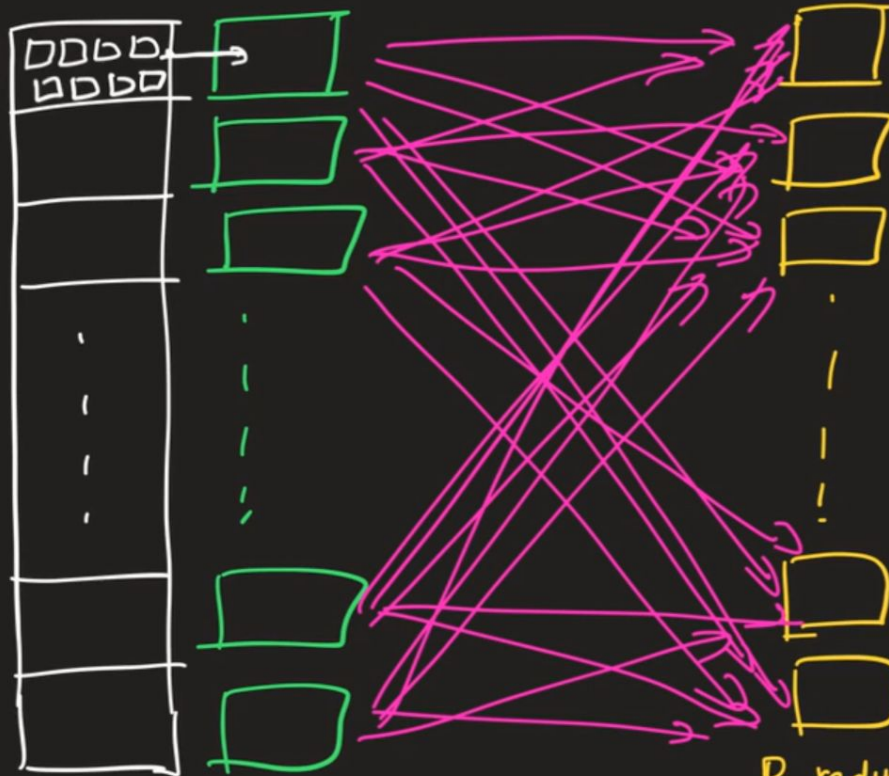
SS2 13 Map Reduce



SS2 14 Map Reduce scheduling

How does map-reduce speed up the batch processing pipeline?

64MB



M mappers

50 billion docs

1000 words

6 char

$1000 \times 6 \sim 6000$

6 KB char

Space per doc

$$\frac{50 \times 10^9 \times 6 \times 10^3}{1}$$

$$= 300 \times 10^{12}$$

$$= 300 \text{ TB}$$



10ms

5ms

→ Seek time

→ rotational delay

data transfer time

R reducers

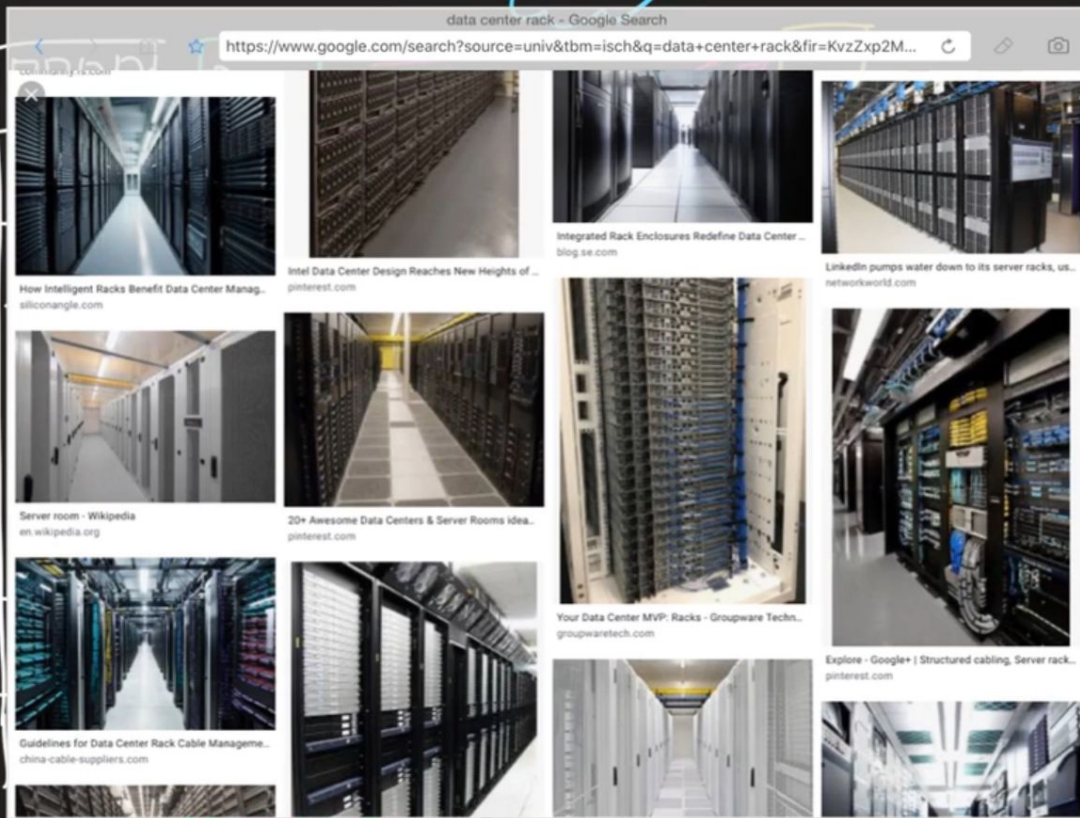
$$\frac{300 \text{ TB}}{50 \text{ MB/sec}} = 6 \times 10^6 \text{ sec}$$

$$= 70 \text{ days}$$

64MB



Scheduler



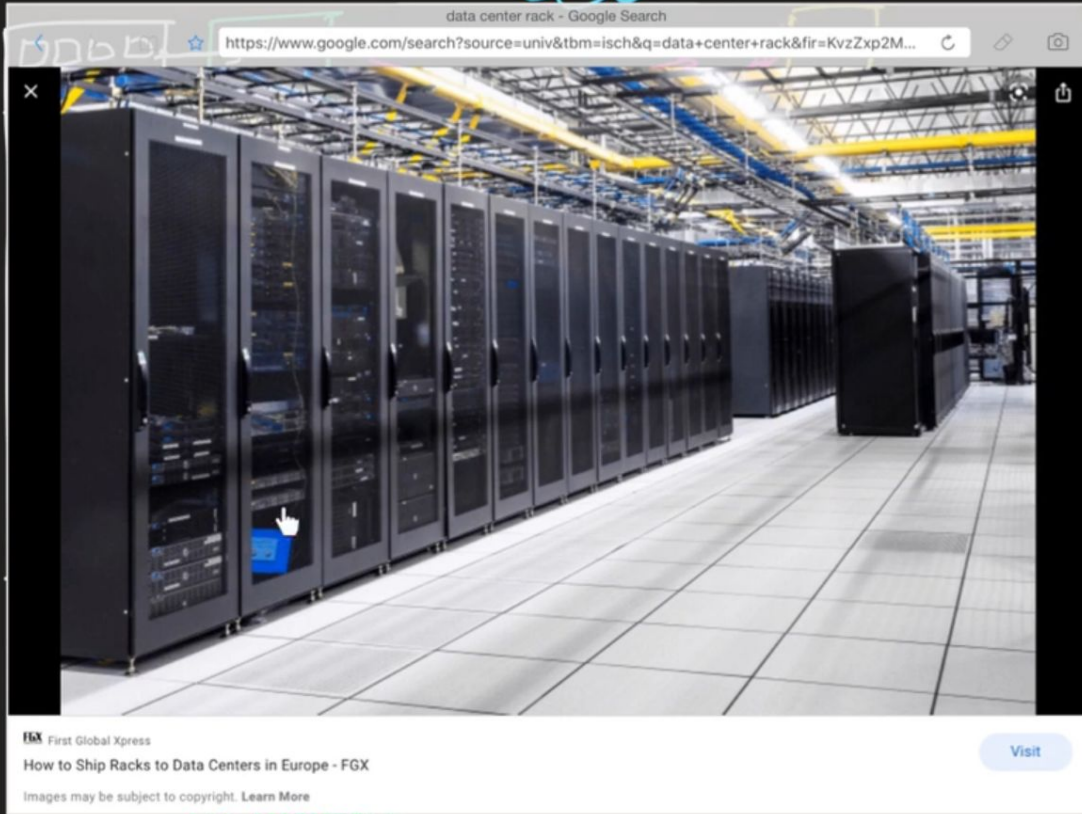
IM mappers

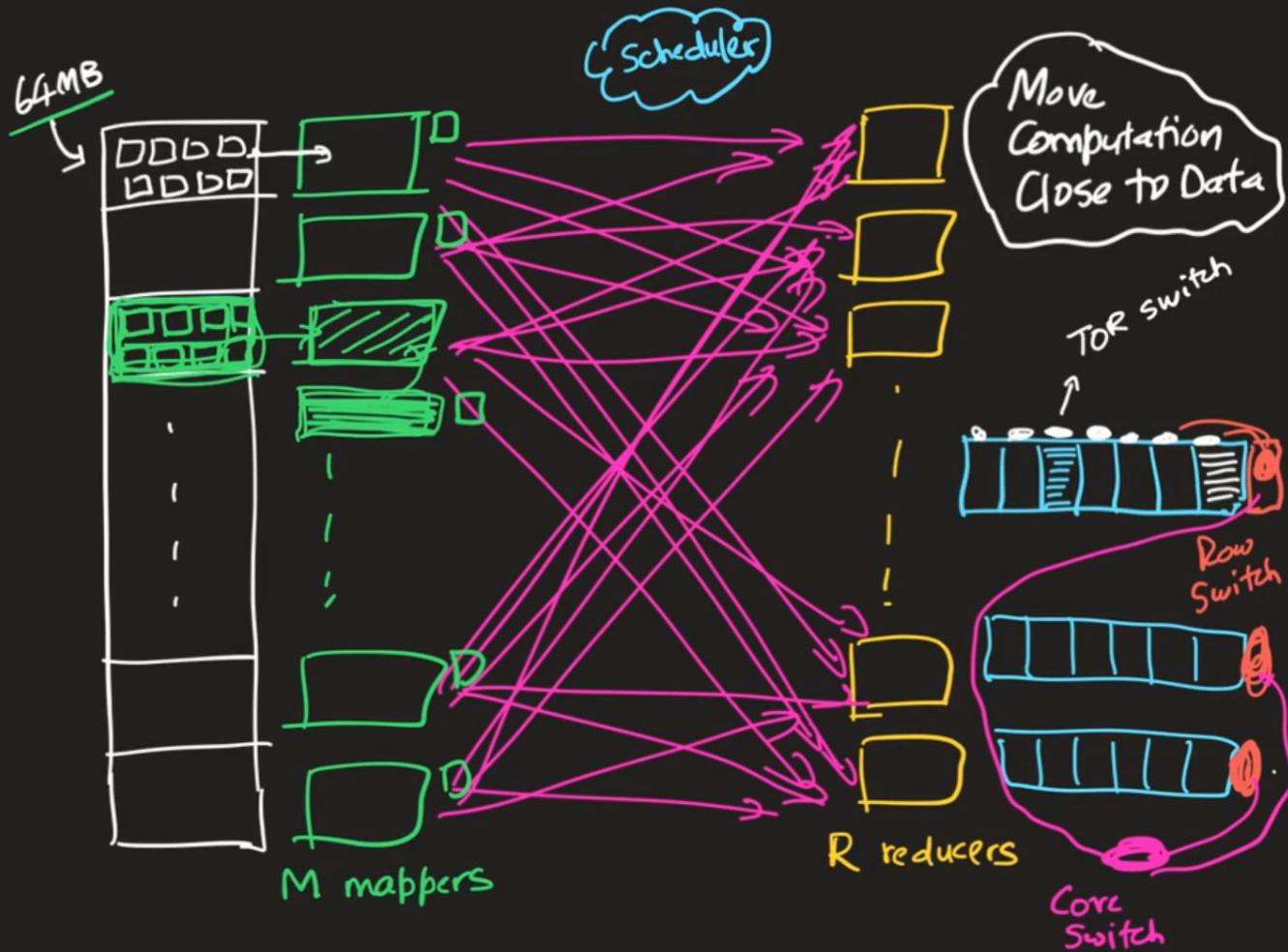
{ik}

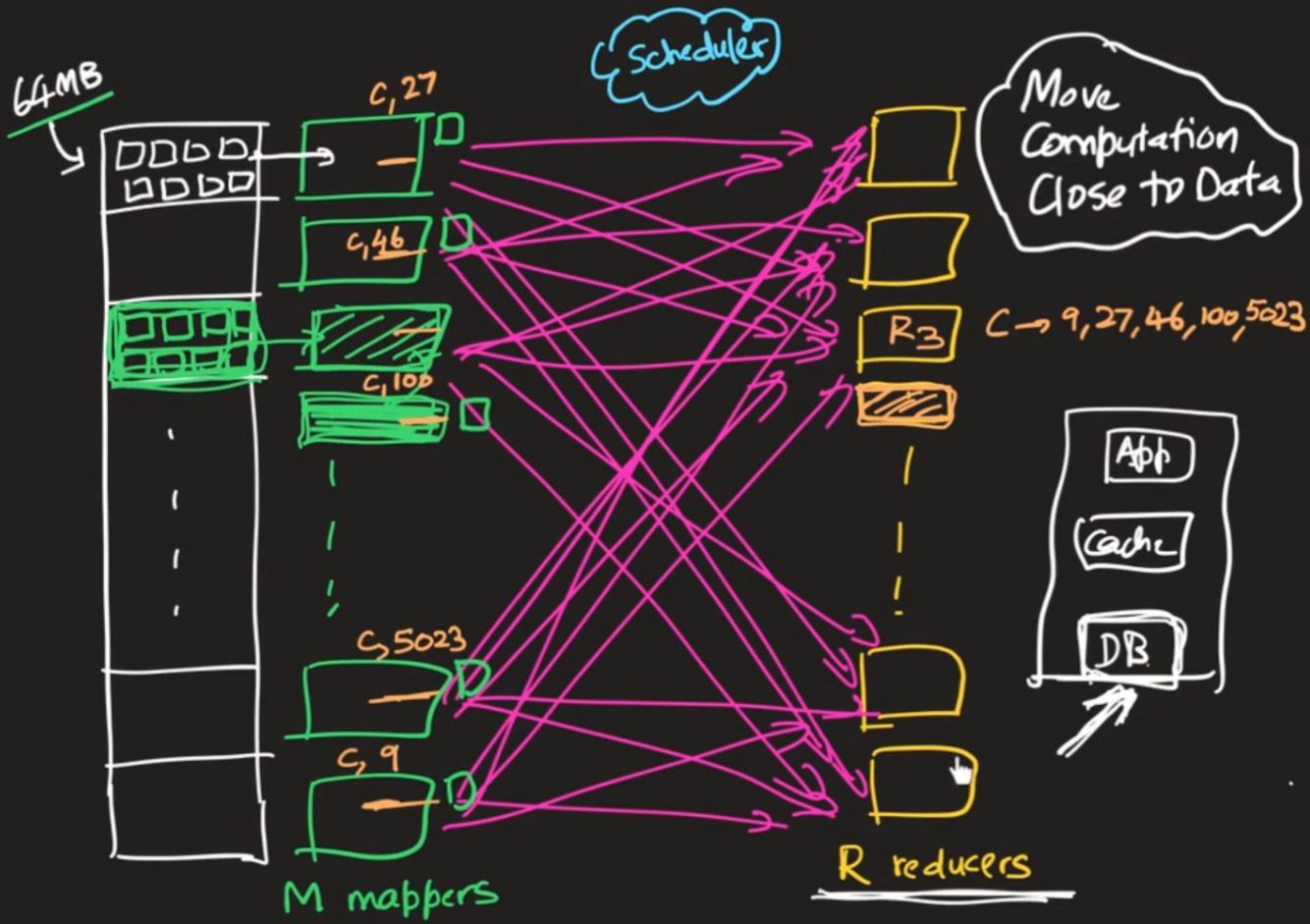
64MB



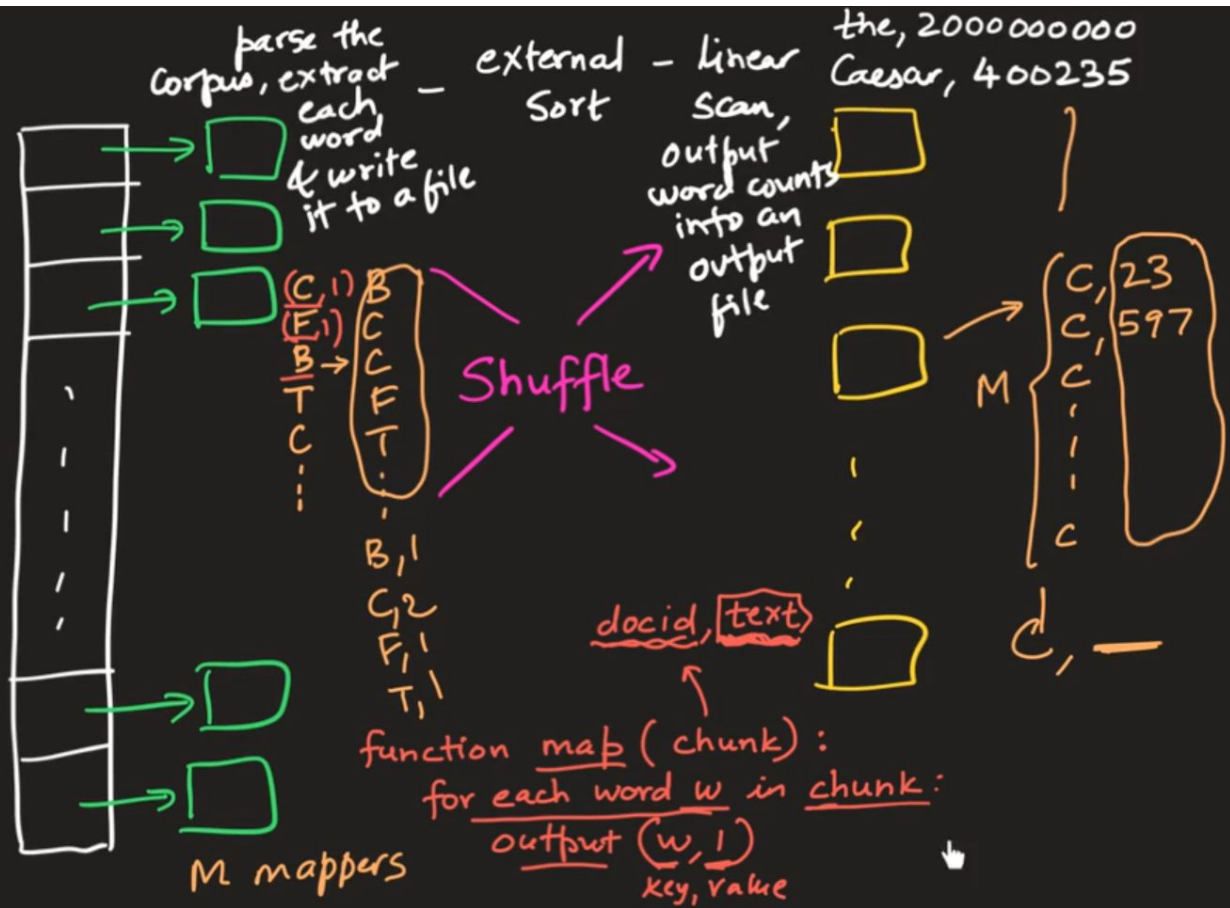
Scheduler

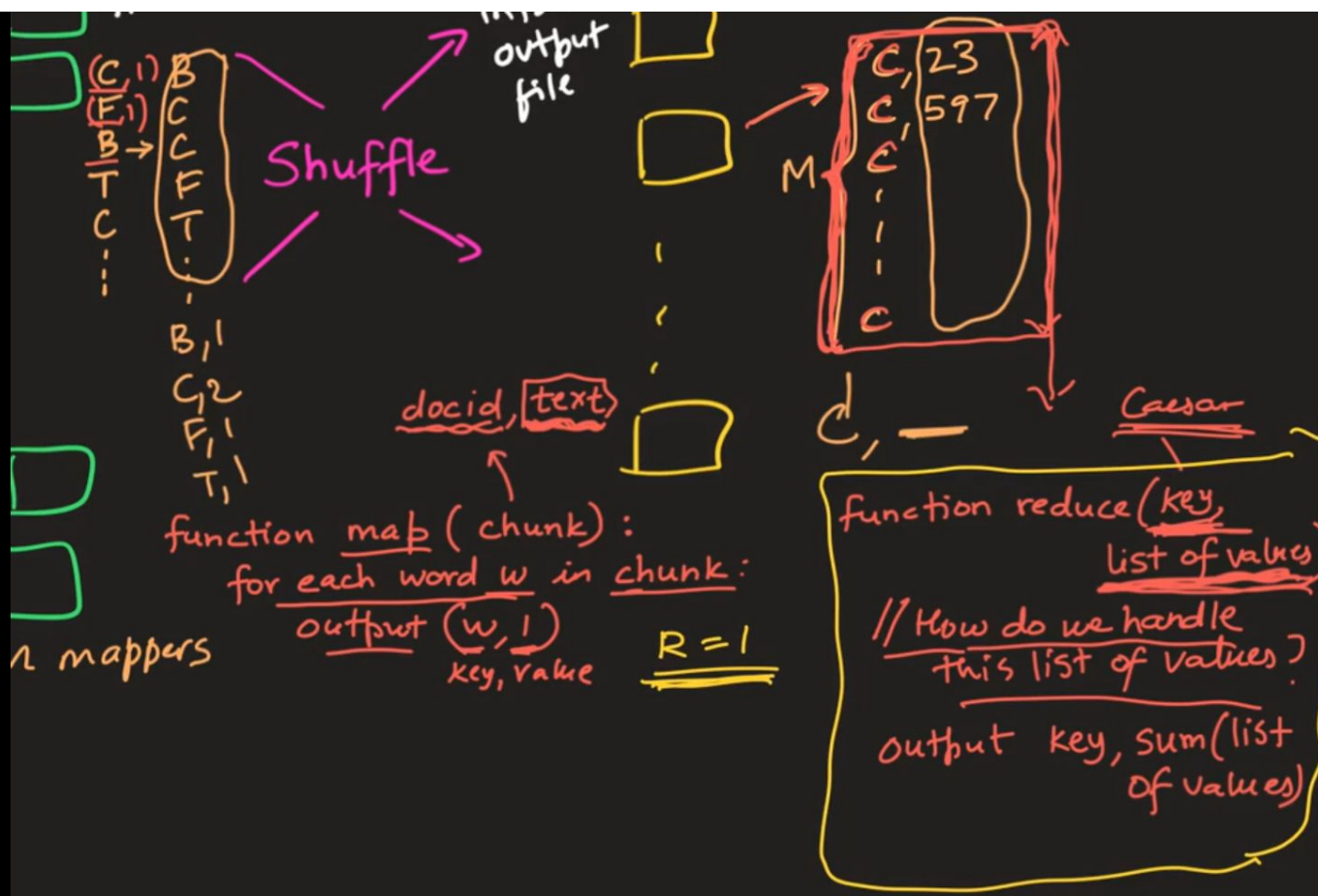






SS2 15 WordCount





LISP
map square [1,2,3,4] → [1,4,9,16]
reduce
⊕
[1,4,9,16]

parse the corpus, extract each word & write it to a file
external Sort
linear Scan,
output word counts into an output file

the, 2000000000
Caesar, 400235



(C,1)
(F,1)
B
T
C
...

Shuffle

Combine function
B,1
C,2
F,1
T,1

docid, text

function map(chunk):
for each word w in chunk:

output (w, 1)
key, value

R=1

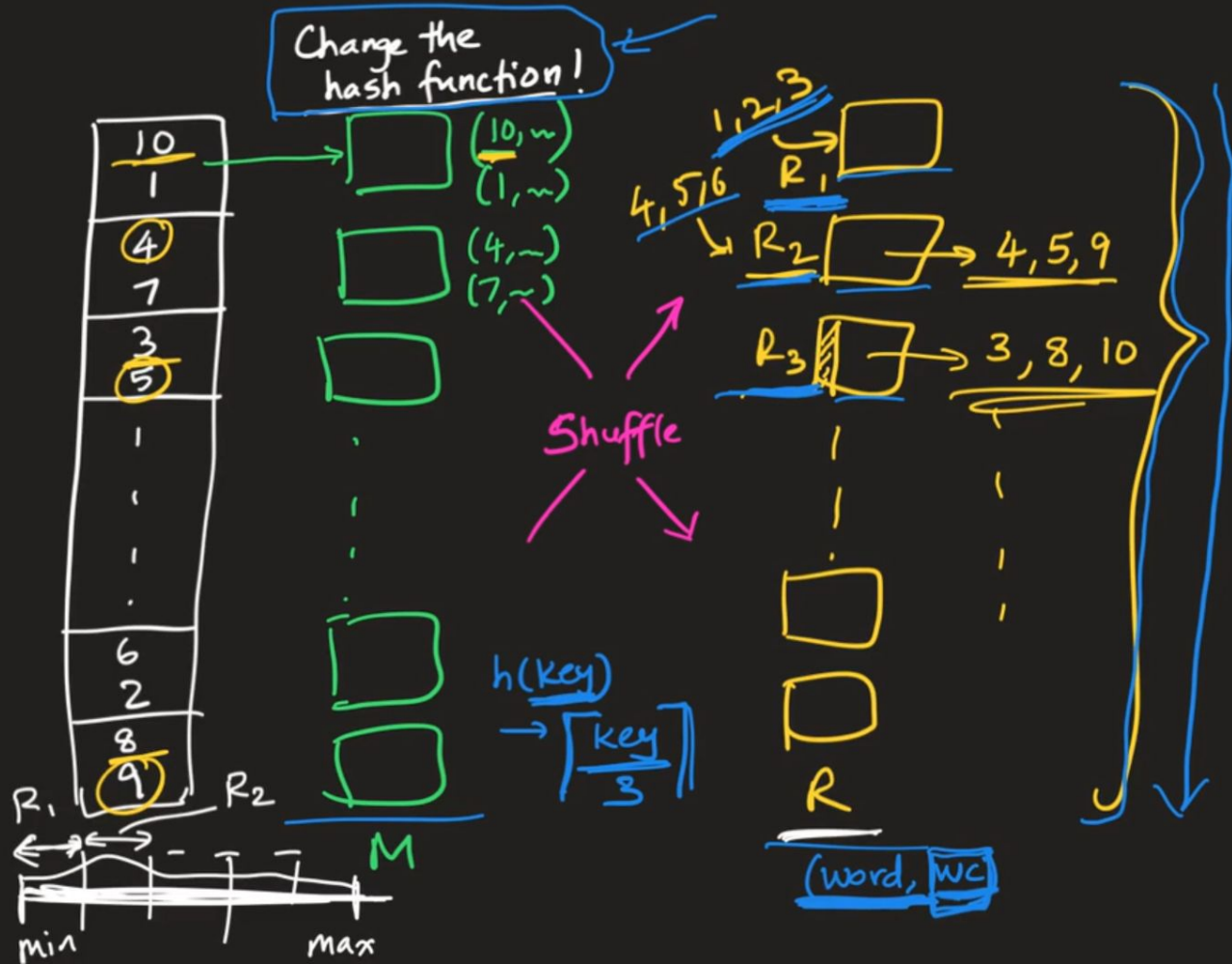
M mappers



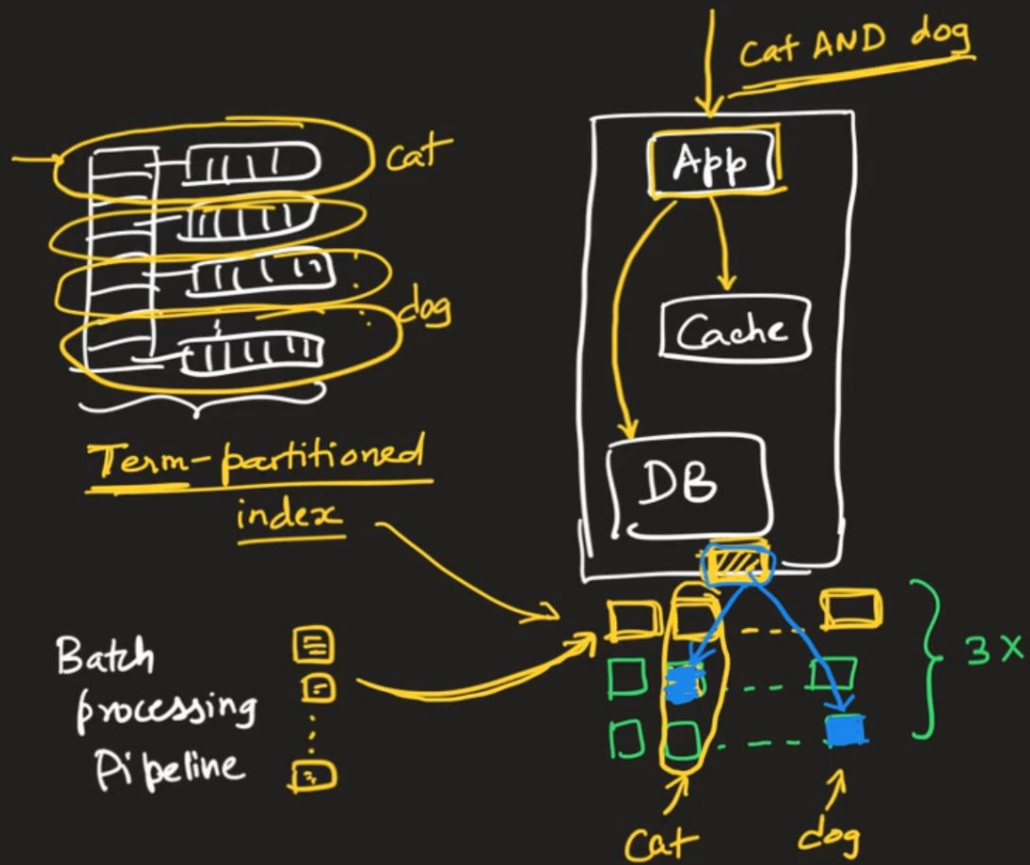
C, 23
C, 597
C
...

function
// How this
output

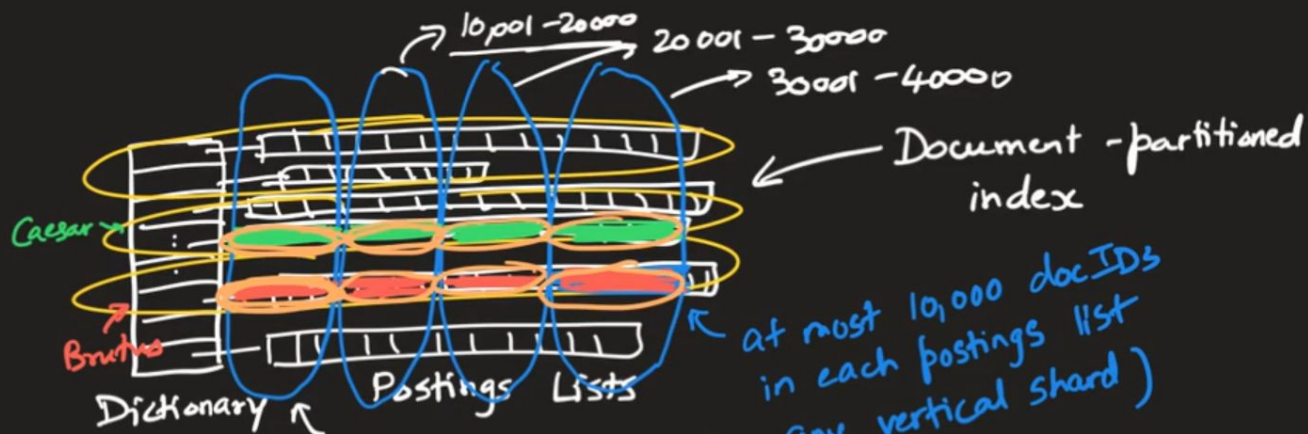
SS2 16 Distributed Sorting



SS2 17 Term-partitioned Index



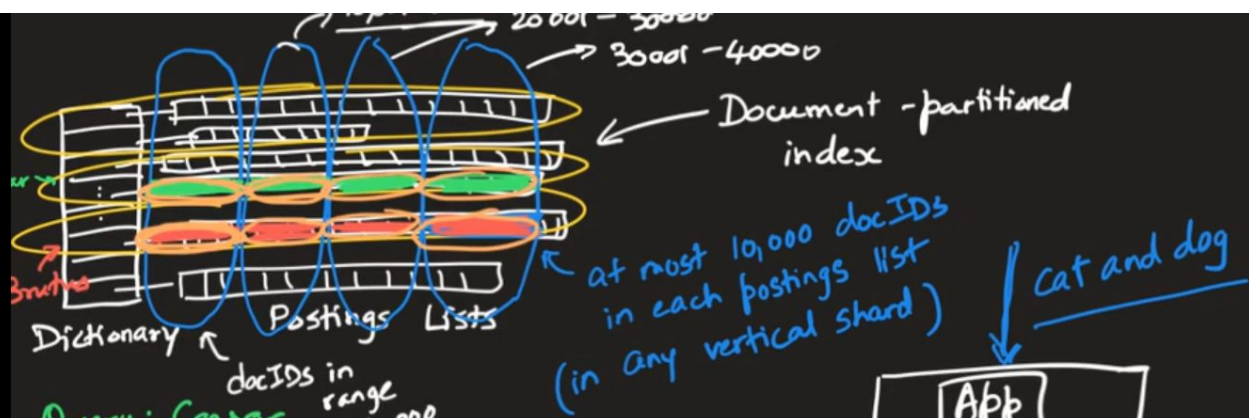
SS2 18 Document-partitioned Index with Scatter-Gather for Parallelism



Dictionary
Postings Lists
docIDs in range 1-10,000
Query: Caesar

Query: Caesar AND Brutus
OR

NOT Caesar



Query: Caesar docIDs in range 1-10,000

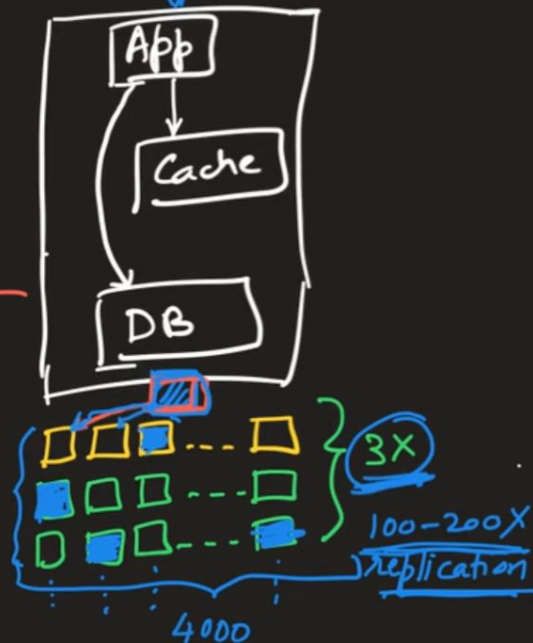
Query: Caesar AND Brutus

OR

NOT Caesar

Scatter/Gather

Batch processing pipeline



Query: Caesar AND Brutus
OR
NOT Caesar

Hash function:

$$= \left\lceil \frac{\text{docID}}{10,000} \right\rceil$$



(term, docID)

10,001-20,000

Shuffle

Scatter/Gather

